CampusTrade — Entregable Final de Desarrollo, Repositorio y Despliegue — Equipo 3 MEX-PER

Instituto Tecnológico Superior del Oriente del Estado de Hidalgo

Ingeniería en Sistemas Computacionales · Equipo 3 MEX-PER

Integrantes confirmados: Alexis Martin Cortés De Lucio y Brandon Joshua Labastida Jiménez.

Fecha de documentación: 30 de octubre de 2026. Versión candidata: 0.3.0-rc.1.

Repositorio: https://github.com/JoshStra/Compraventa_Peru_Mexico

Índice

Introducción · Objetivo y alcance · Metodología · 1. Backlog · 2. Issues · 3. Incidencias · 4. Estructura · 5. Colaboración · 6. PR · 7. Release y despliegue · Conclusiones · Anexos.

Introducción y objetivo

CampusTrade permite que estudiantes publiquen y encuentren materiales para venta, renta, intercambio y donación. Esta entrega transforma el prototipo local en una aplicación React/Express con PostgreSQL compartido. El objetivo es demostrar funcionalidad real, validaciones, trazabilidad de cambios y evidencia académica verificable, distinguiendo el código probado de las activaciones externas pendientes.

Alcance y metodología

Se contrastaron Especificación Técnica, Fase 2, Entregable 4 y Validación técnica del diseño aportados por el usuario. Sus descripciones se trataron como referencias de requisitos, sin dar por implementados sus diagramas propuestos. Se auditó main, se conservaron origin e historial y se integraron cambios en incrementos funcionales con pruebas antes del commit. Cada requisito enlaza código, prueba y SHA. Las fechas de Git se conservaron; no se simularon autores, revisiones ni despliegues.

Las pruebas usan datos sintéticos y PostgreSQL local real. El control del dominio itesa.edu.mx está configurado, pero el envío real queda pendiente por decisión explícita del responsable. Los límites de planes gratuitos se contrastaron con fuentes oficiales el 30-10-2026. No se contrataron servicios de pago.

1. Avance del backlog

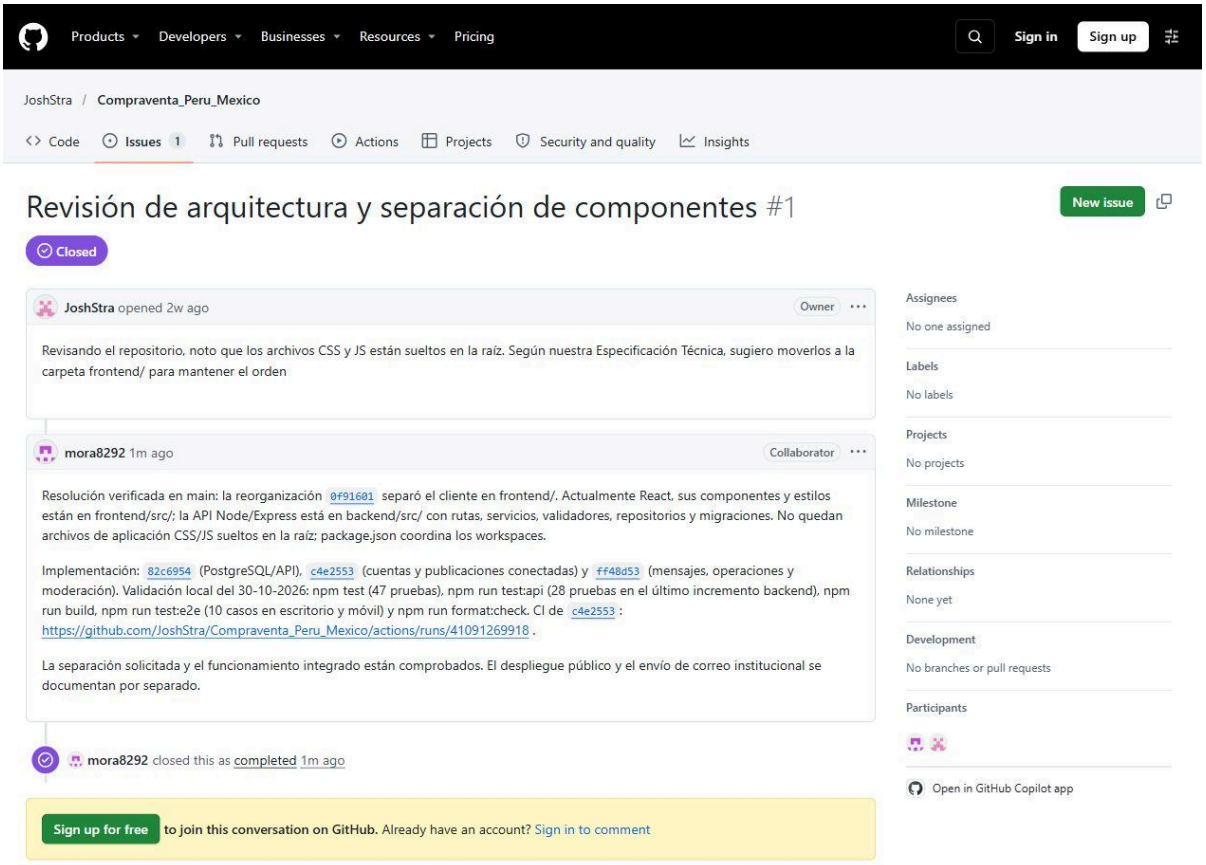
El apartado 1 incluye catorce historias RF-01 a RF-14, prioridades, criterios, dependencias, responsable verificable, fechas reales y commits. Todos tienen código. La verificación institucional completa necesita correo real; las imágenes productivas requieren S3 externo probado. RF-05 tiene flujo común de donación pero falta un recorrido dedicado. Por ello no se usa un porcentaje arbitrario de cumplimiento total.

Hitos reales: auditoría 09:49, API/PostgreSQL 09:52, cuentas 09:58, publicaciones 10:04, mensajes 10:08, operaciones 10:12, moderación 10:15, interfaz 10:35 y 10:47, correcciones 10:51 del 30 de octubre (UTC−06). El trabajo externo siguiente no tiene una fecha acordada con el curso.

2. Issues cerrados

El issue #1 confirma la separación frontend/backend pedida por JoshStra. El comentario de mora8292 detalla archivos y pruebas; cierre real 16:49:31Z. El issue #2 documenta dos regresiones reproducidas y corregidas en 6b0d297: cancelar solicitudes al retirar una oferta y serializar fechas civiles de renta. Cierre real 16:52:06Z. Ambos tienen comentario técnico real y evidencia de pruebas. El formulario completo está en apartado 2.

Figura 1. Issue de arquitectura cerrado con comentario, captura auténtica de GitHub. No representa una aprobación por PR.

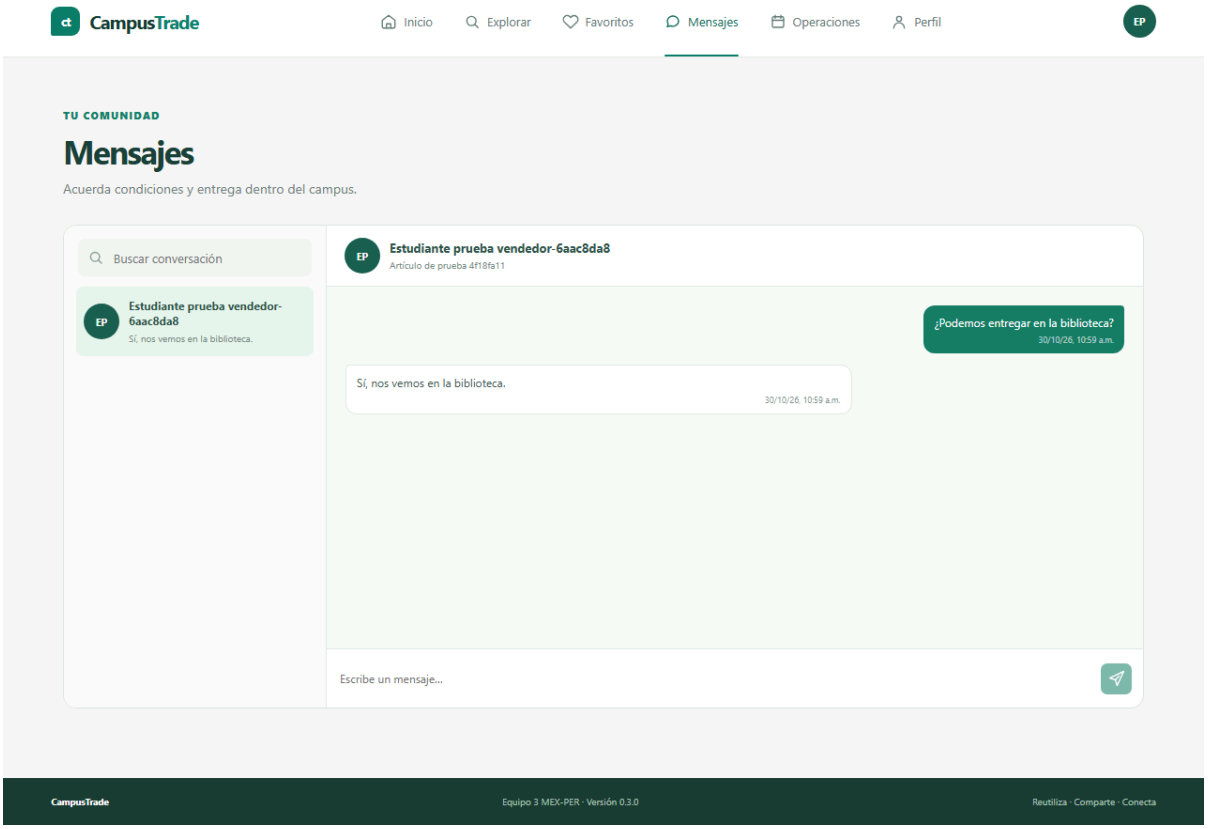


3. Logs de incidencias y soluciones

La bitácora registra fallos de formato en CI, lectura de imágenes bajo .data, enlace hash de verificación, selectores ambiguos de pruebas y los dos defectos del issue #2. No se ocultan ejecuciones fallidas: antes de corregir operaciones pasaron 27 de 29 pruebas; después pasaron treinta incluyendo reinicio de procesos. El apartado 3 detalla reproducción, severidad, causa, solución y estado.

Validación final: 47 pruebas frontend (44 de regresión local y tres de cliente HTTP), 30 API/PostgreSQL y diez E2E (cinco recorridos en escritorio y móvil), compilación y formato. Las entradas inválidas de dominio, roles, campos vacíos, precios, imágenes, acceso ajeno y calificación anticipada se rechazan. Los logs de evidencias contienen las salidas reales y fecha de ejecución.

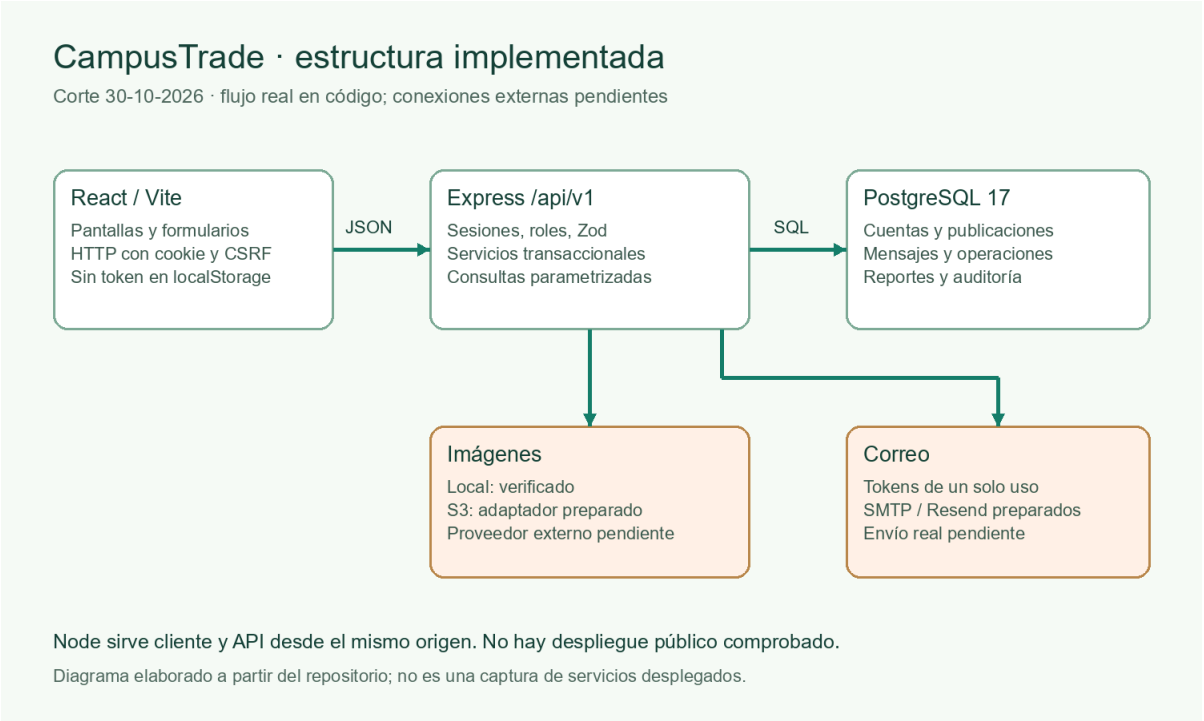
Figura 2. Conversación entre dos usuarios sintéticos. El navegador y la API usan sesiones distintas y la información persiste en PostgreSQL local.



4. Estructura funcional

React → HTTP JSON con cookie/CSRF → Express → middleware de identidad y permisos → servicios → PostgreSQL. Las imágenes se guardan fuera de la base; el correo utiliza un adaptador. El servidor sirve frontend y API desde el mismo origen. Las migraciones 001/002 crean relaciones, rangos y unicidad; las reservas bloquean filas dentro de una transacción. El apartado 4 contiene el árbol real, diagrama de componentes y ER, enlazados a las migraciones.

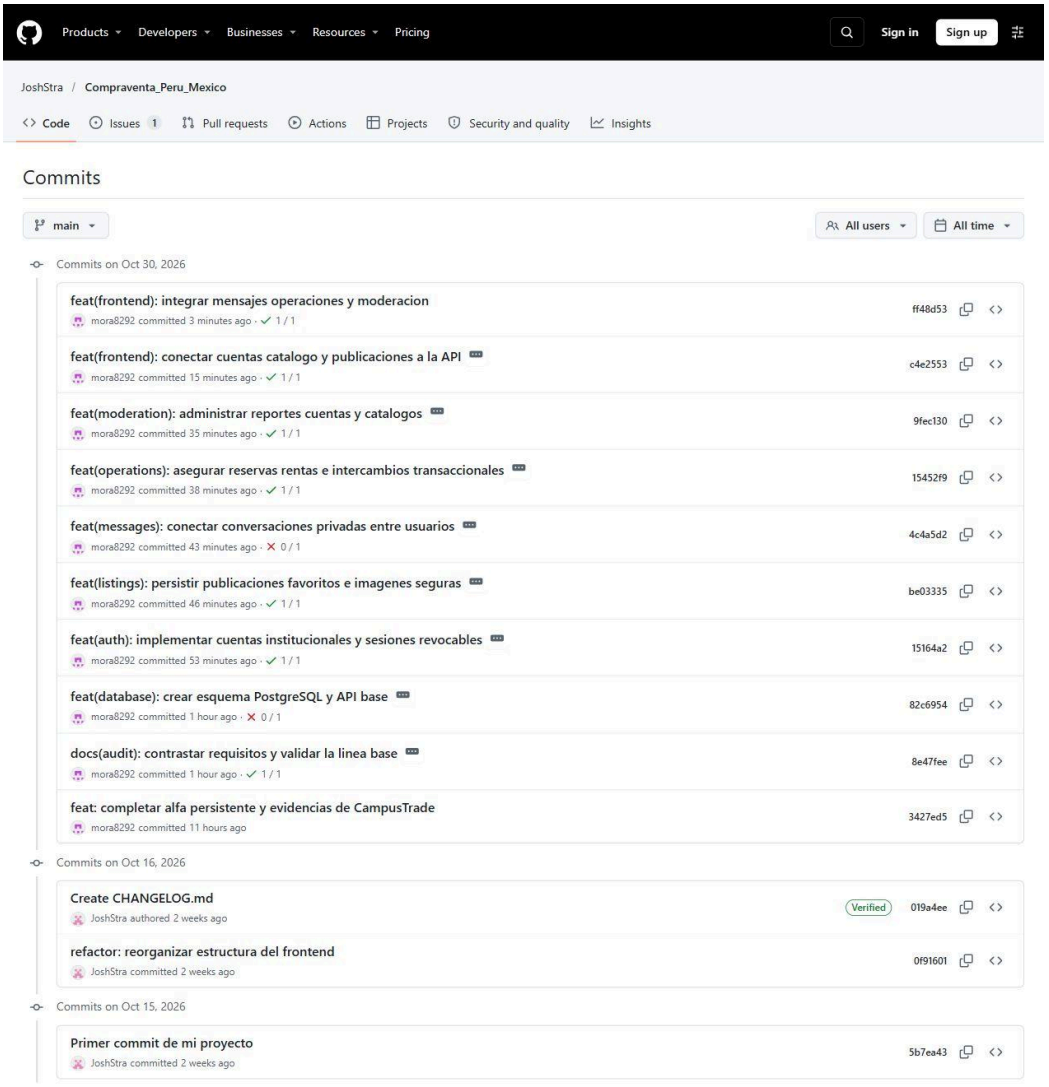
La mensajería comprueba participación; la operación deriva identidad de la sesión; la calificación exige finalización. La moderación conserva auditoría y el administrador se promueve por CLI sobre cuenta activa verificada. No hay administrador predeterminado. Los listados recientes tienen límites documentados y no se afirman microservicios ni servicios remotos desplegados.



5. Uso activo del repositorio colaborativo

Se preservó https://github.com/JoshStra/Compraventa_Peru_Mexico y main. Brandon Labastida figura como autor de la base/reorganización; JoshStra creó CHANGELOG y abrió el issue #1; mora8292 registró los incrementos actuales y las resoluciones. Las cuentas GitHub contribuyentes observadas son JoshStra y mora8292. La conversación del issue muestra coordinación real, aunque los incrementos recientes proceden de una sola cuenta. No se atribuye el trabajo a personas sin evidencia. El apartado 5 conserva SHAs y fechas.

Figura 3. Historial real de commits. No se modificaron horarios ni se agregaron commits vacíos.



6. Pull requests revisados

La consulta de todos los PR devolvió cero resultados. El desarrollo directo en main estaba autorizado. No existen revisiones, revisores o aprobaciones que inventariar: criterio PENDIENTE de una revisión independiente real. La comprobación adicional de issues cerrados sí confirma estado completed, comentario, commit y pruebas en #1/#2. Véase apartado 6.

7. Releases y despliegue

La candidata v0.3.0-rc.1 refleja la primera integración full stack y conserva limitaciones explícitas. El tag apunta a addc154 y el release se publicó el 30-10-2026, 17:07:25Z; su enlace está en el apartado 7. Una publicación de fuentes no prueba un despliegue.

Se preparó Render Free para Node y frontend del mismo origen, Neon Free para PostgreSQL y un adaptador S3 compatible. La alternativa Cloudflare Pages se evaluó, pero separar el cliente complicaba cookies/CSRF. Render Free suspende por inactividad y su disco es efímero; su PostgreSQL gratuito caduca a treinta días, por lo que no se eligió. El correo SMTP estándar está bloqueado allí. La documentación detalla cuotas de Neon y una alternativa S3 gratuita sin afirmar que se haya conectado.

No hay frontend/backend públicos comprobados: faltan cuentas conectadas y secretos externos. El correo permanece pendiente por decisión del responsable. No existe captura de aplicación desplegada. Apartado 7 y procedimiento.

Matriz final de la rúbrica

Criterio	**Estado**	**Justificación / evidencia**
1. Avance del backlog	CUMPLIDO	RF-01 a RF-14, historias, dependencias, criterios, responsables verificables y cronograma real en 01.
2. Issues cerrados	CUMPLIDO	#1 y #2 cerrados con comentario técnico, commit y regresiones; 02 y capturas.
3. Logs e incidencias	CUMPLIDO	Bitácora, fallos reales y validaciones positivas/negativas; 03 y logs.
4. Estructura funcional	CUMPLIDO	Árbol real, componentes, flujo y ER de las migraciones; 04.
5. Repositorio colaborativo	CUMPLIDO	Origin compartido, dos cuentas contribuyentes y conversación verificable; 05. No acredita revisión independiente.
6. PR revisados	PENDIENTE	Cero PR; revisión real de otro integrante por realizar. La comprobación de issues sí está completa.
7. Releases y despliegue	PARCIAL	Candidata probada y configuración de publicación; sin aplicación pública verificada. Release de fuentes no equivale a despliegue.

Conclusiones

CampusTrade dispone de cliente, API y persistencia compartida funcionales y probados localmente. La evidencia conecta requisitos con código, pruebas, commits e issues. Permanecen pendientes la revisión real por PR, los accesos para publicación y la activación de correo/almacenamiento externos. Es una candidata verificable de fuentes; el entorno no permite declarar una entrega productiva completamente desplegada.

Anexos y referencias

Los siete apartados amplían el contenido de este documento. Los PNG desktop/mobile muestran localhost con fixtures campus.test; los JPG muestran GitHub auténtico. No se capturaron contraseñas ni tokens. Archivos técnicos: API.md, DATABASE.md, ARCHITECTURE.md, SECURITY.md, TESTING.md, TRACEABILITY.md y RELEASE_CHECKLIST.md en docs.

Documentos de referencia: CampusTrade_Especificacion_Tecnica_ISC_Equipo_3_MEX_PER.docx (1).pdf; CampusTrade_Fase2_Sistemas_ISC_Equipo_3_MEX_PER.docx (1).pdf; ENTREGABLE 4. REPOSITORIO (2) (1).pdf; 7. Validación técnica del diseño (1).pdf. Las conclusiones actuales se basan en implementación y pruebas, no en el estado histórico descrito en esos PDF.

Referencias de despliegue: Render Free, Neon Free, Supabase S3. Referencias de controles: Express, transacciones node-postgres, OWASP contraseñas.

Resumen de pruebas

Comando	Resultado	Alcance
npm test	47 correctas	44 regresiones locales y 3 del cliente HTTP
npm run test:api	30 correctas	PostgreSQL, permisos, concurrencia y reinicio Node
npm run test:e2e	10 correctas	5 recorridos en escritorio y Pixel 7
npm run build	Correcto	Cliente HTTP productivo Vite
npm run format:check	Correcto	Formato del repositorio
npm audit --omit=dev	0 reportadas	Dependencias de ejecución al 30-10-2026